Lecture 9. Classification II

Thresholds, confusion matrix, precision and recall, ROC and AUC, separability and regularized logistic regression

Contents

9.1 Logistic regression as a classifier **2**
- 9.1.1 Thresholding . . . 2
- 9.1.2 Accuracy . . . 2

9.2 Evaluating classifiers **3**
- 9.2.1 Drawbacks of accuracy . . . 3
- 9.2.2 The confusion matrix . . . 3
- 9.2.3 ROC and precision–recall curves . . . 4
- 9.2.4 Training versus testing . . . 4

9.3 Linear separability and regularization **5**

9.1 Logistic regression as a classifier

Motivation. A logistic model gives probabilities; turning them into decisions requires a threshold, and judging the decisions requires more than the share of correct answers. This lecture is about evaluating classifiers honestly, especially when one class is rare.

Review: logistic regression predicts $P(y = 1 \mid x) = \operatorname{expit}(x^\top \beta)$, is fitted with cross-entropy loss, and classifies $y = 1$ when $\hat{p} \geq 0.5$.

A coin example. Ten i.i.d. tosses give $0, 0, 0, 1, 1, 0, 0, 0, 1, 1$. The best constant model $P(y = 1) = \theta$ is the sample mean 0.4, because it maximises the likelihood $\theta^4(1-\theta)^6$, i.e. $4 \log \theta + 6 \log(1-\theta)$, whose derivative $4/\theta - 6/(1-\theta)$ vanishes at $\theta = 0.4$; equivalently it minimises the cross-entropy loss.

The null model. The slides use the census data (24% high earners). We continue with `ISLR2::Default` (credit-card default, about 3% "Yes"), split 80/20 as in Lecture 8. The null model $P(y = 1) = \operatorname{expit}(\beta_0)$ predicts the same probability $\bar{\pi}$ for everyone; since $\bar{\pi} < 0.5$, everyone is classified as $y = 0$.

```
set.seed(364)
idx <- sample(nrow(Default), 0.8 * nrow(Default))
train <- Default[idx, ]; test <- Default[-idx, ]
train %>% count(default) %>% mutate(pct = n / sum(n))
```

```
##   default    n     pct
## 1      No 7750 0.96875
## 2     Yes  250 0.03125
```

9.1.1 Thresholding

The decision rule need not use 0.5:

$$\text{classify}(x) = \begin{cases} 1, & \hat{P}(y = 1 \mid x) \geq t, \\ 0, & \text{otherwise.} \end{cases}$$

A logistic regression paired with a threshold is a classifier. Raising t classifies fewer observations as 1: fewer false positives but more false negatives.

9.1.2 Accuracy

The simplest evaluation is **accuracy**, the proportion of observations classified correctly. Comparing the null model, a model with `balance` only, and a model with all predictors:

```
fit_null <- glm(default ~ 1, family = binomial, data = train)
fit_1 <- glm(default ~ balance, family = binomial, data = train)
fit_all <- glm(default ~ student + balance + income, family = binomial, data = train)
classify <- function(fit, data, t = 0.5)
  factor(ifelse(predict(fit, data, type = "response") >= t, "Yes", "No"), levels = c("No
    ", "Yes"))
map_dbl(list(null = fit_null, bal = fit_1, all = fit_all),
        ~ mean(classify(.x, train) == train$default))
```

```
##     null      bal      all
## 0.968750 0.974125 0.974875
```

Adding predictors improves training accuracy, but only slightly, because the null model is already 97% accurate.

9.2 Evaluating classifiers

9.2.1 Drawbacks of accuracy

A spam filter applied to 100 e-mails, 2 spam and 98 ham, that labels everything as ham is 98% accurate and useless. With imbalanced classes accuracy is misleading.

9.2.2 The confusion matrix

Call class 1 positive and class 0 negative.

	predicted 1	predicted 0
actual 1	TP (true positive)	FN (false negative: missed)
actual 0	FP (false positive: false alarm)	TN (true negative)

$$\text{accuracy} = \frac{TP+TN}{n}, \qquad \text{precision} = \frac{TP}{TP+FP}, \qquad \text{recall} = \frac{TP}{TP+FN}.$$

Precision: of the observations predicted 1, the proportion that are actually 1. **Recall**: of the actual 1s, the proportion predicted 1. For the all-ham spam filter, TP = FP = 0, so precision is undefined (0/0) and recall is $0/2 = 0$.

```
table(predicted = classify(fit_1, test), actual = test$default)
```

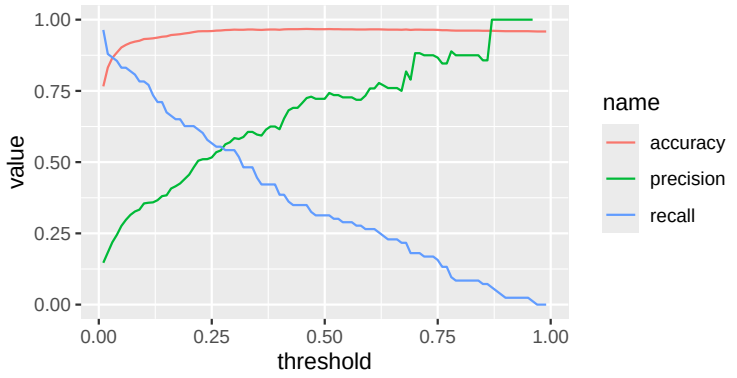
```
##          actual
## predicted   No  Yes
##       No  1907   58
##       Yes   10   25
```

The trade-off. Precision penalises false positives and recall penalises false negatives. A threshold of 0 (classify everyone as 1) gives 100% recall but low precision. Higher thresholds give fewer FP and higher precision; lower thresholds give fewer FN and higher recall. Accuracy is typically highest in between.

Choosing a metric depends on the domain. For screening a disease, maximise recall: a false negative means no treatment, while a false positive only leads to further tests. For deciding a life sentence, maximise precision: a false positive puts an innocent person in prison.

Other names: **sensitivity** = recall = true positive rate (TPR); **specificity** = true negative rate $= TN/(TN+FP)$; 1−specificity = false positive rate (FPR); precision = positive predictive value (PPV).

```
p_test <- predict(fit_all, test, type = "response")
y_test <- test$default == "Yes"
tibble(t = seq(0.01, 0.99, by = 0.01)) %>%
  mutate(accuracy = map_dbl(t, ~ mean((p_test >= .x) == y_test)),
         precision = map_dbl(t, ~ sum(p_test >= .x & y_test) / sum(p_test >= .x)),
         recall = map_dbl(t, ~ sum(p_test >= .x & y_test) / sum(y_test))) %>%
  pivot_longer(-t) %>%
  ggplot(aes(t, value, colour = name)) + geom_line(na.rm = TRUE) + labs(x = "threshold")
```



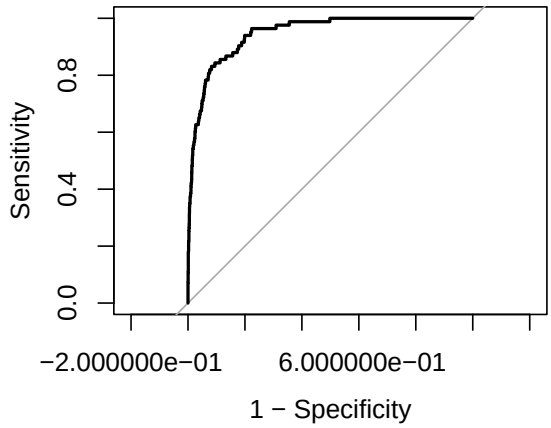
9.2.3 ROC and precision–recall curves

Plotting precision against recall over all thresholds gives the **precision–recall (PR) curve**; the perfect classifier reaches precision 1 and recall 1. Plotting sensitivity (TPR) against 1−specificity (FPR) gives the **ROC** (receiver operating characteristic) curve, standard in machine learning, operations research and medical testing. We want the ROC curve as close to the top-left corner as possible; the **area under the curve** (AUC) summarises it: 1 for a perfect classifier, 0.5 for random guessing. The PR curve is more informative when the classes are highly unbalanced.

```
roc_1 <- roc(test$default, predict(fit_1, test, type = "response"), quiet = TRUE)
roc_all <- roc(test$default, p_test, quiet = TRUE)
c(auc_balance = auc(roc_1), auc_all = auc(roc_all))
```

```
## auc_balance     auc_all
##   0.9468359   0.9467919
```

```
plot(roc_all, legacy.axes = TRUE)
```



What AUC measures [Supplement]

AUC equals the probability that a randomly chosen positive case receives a higher predicted probability than a randomly chosen negative case (the Mann–Whitney statistic). It therefore measures ranking quality and does not depend on the threshold or on the class proportions.

9.2.4 Training versus testing

With the census data the slides compare three models:

model	accuracy (train)	accuracy (test)	sensitivity (test)	specificity (test)
all predictors	0.852	0.849	0.598	0.928
capital gain only	0.800	0.803	0.204	0.991
null	0.759	0.761	0.000	1.000

Each model does slightly worse on the test set. The null model has sensitivity 0 and specificity 1, since it always predicts the same class; the full model is a little less specific but much more sensitive, so it is the most useful. For `Default`:

```
eval_model <- function(fit) {
  pr <- classify(fit, test)
  c(acc_train = mean(classify(fit, train) == train$default),
    acc_test = mean(pr == test$default),
    sens = mean(pr[test$default == "Yes"] == "Yes"),
    spec = mean(pr[test$default == "No"] == "No"))
}
round(rbind(all = eval_model(fit_all), balance = eval_model(fit_1), null = eval_model(
    fit_null)), 3)
```

```
##         acc_train acc_test  sens  spec
## all         0.975    0.967 0.313 0.995
## balance     0.974    0.966 0.301 0.995
## null        0.969    0.959 0.000 1.000
```

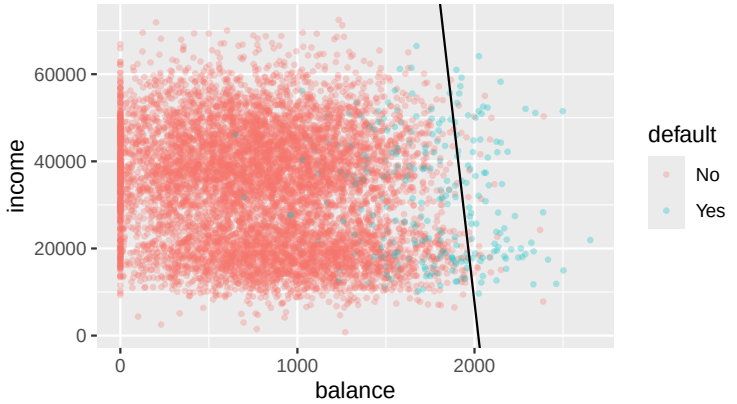
At threshold 0.5 the sensitivity is low: most defaulters are missed. If missing a defaulter is costly, a lower threshold is appropriate.

9.3 Linear separability and regularization

Motivation. With one predictor the decision boundary is a point, with two a line. When a line separates the classes perfectly, maximum likelihood breaks down: the coefficients run off to infinity. Regularization fixes this.

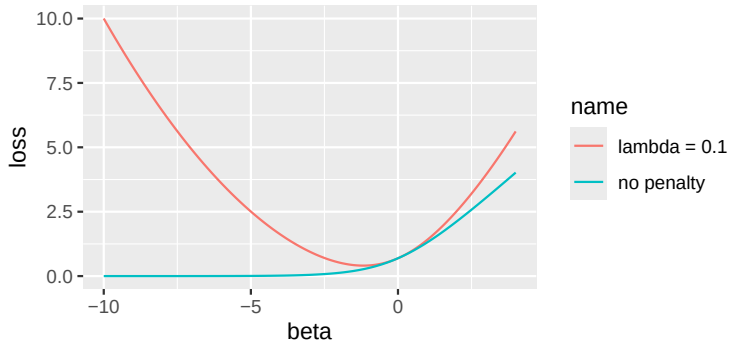
Decision boundaries. With hours worked per week as the only predictor, the slides obtain $\operatorname{logit}\hat{\pi} = -3.085 + 0.046 \cdot \text{hours}$; at threshold 0.5 the boundary is $-3.085 + 0.046 \cdot \text{hours} = 0$, i.e. about 67 hours. With two predictors, $\hat{\pi} = \operatorname{expit}(-4.85 + 0.047 \cdot \text{hours} + 0.042 \cdot \text{age})$, the boundary is a line in the (hours, age) plane; points of class 1 on the class-0 side are false negatives and vice versa.

```
fit_2 <- glm(default ~ balance + income, family = binomial, data = train)
b <- coef(fit_2)
ggplot(train, aes(balance, income, colour = default)) +
  geom_point(alpha = 0.3, size = 0.8) +
  geom_abline(intercept = -b[1] / b[3], slope = -b[2] / b[3])
```



Perfect separation. Take two points, $(x, y) = (-1, 1)$ and $(1, 0)$, and the model $\text{expit}(\beta_1 x)$. The mean cross-entropy loss decreases monotonically to 0 as $\beta_1 \to -\infty$, so the "best" estimate is $\hat{\beta}_1 = -\infty$, which predicts probabilities of exactly 0 or 1. That is overconfident: a new observation $(0.5, 1)$ would get predicted probability 0 and *infinite* loss.

```
x <- c(-1, 1); y <- c(1, 0); expit <- function(z) 1 / (1 + exp(-z))
ce <- function(b, lambda = 0)
  -mean(y * log(expit(b * x)) + (1 - y) * log(1 - expit(b * x))) + lambda * b^2
tibble(beta = seq(-10, 4, by = 0.05)) %>%
  mutate(`no penalty` = map_dbl(beta, ce), `lambda = 0.1` = map_dbl(beta, ce, lambda =
    0.1)) %>%
  pivot_longer(-beta) %>%
  ggplot(aes(beta, value, colour = name)) + geom_line() + labs(y = "loss")
```



Points are **linearly separable** if a line (in d dimensions, a $(d-1)$-dimensional hyperplane) separates the classes perfectly. Then some coefficients diverge; in practice the software stops at excessively large values (and `glm()` warns that fitted probabilities 0 or 1 occurred).

Regularized logistic regression adds a penalty, e.g. L_2:

$$-\frac{1}{n}\sum_{i=1}^{n}\left\{y_i \log \text{expit}(x_i^\top \beta) + (1-y_i)\log\left(1-\text{expit}(x_i^\top\beta)\right)\right\} + \lambda\sum_{j=1}^{p}\beta_j^2.$$

The penalised loss (curve above) has a finite minimum even for separable data. As with ridge and lasso, standardise the predictors first. In R, `glmnet(..., family = "binomial")` fits it:

```
X <- scale(model.matrix(default ~ student + balance + income, data = train)[, -1])
cv_fit <- cv.glmnet(X, train$default, family = "binomial", alpha = 0)
coef(cv_fit, s = "lambda.min")
```

```
## 4 x 1 sparse Matrix of class "dgCMatrix"
##                      s1
## (Intercept) -4.83965221
## studentYes  -0.11306543
## balance      1.81973290
## income       0.01566196
```